DevOps Roadmap (2026)

Step-by-step guide to becoming a DevOps Engineer or SRE

1. Learn the Basics

- What is DevOps?
- DevOps vs SRE vs DevSecOps
- DevOps Lifecycle
- Agile and DevOps
- DevOps Mindset & Culture

2. Operating Systems & Networking

- Linux Fundamentals
- Networking Basics (DNS, HTTP/S, TCP/IP, Firewalls)
- SSH, SCP, SFTP
- System Monitoring & Troubleshooting

3. Programming & Scripting

- Bash/Shell Scripting
- Python or Go (choose one)
- Version Control (Git)

4. Infrastructure as Code (IaC)

- Configuration Management (Ansible, Chef, Puppet)
- Provisioning (Terraform, CloudFormation)
- Containerization (Docker)
- Container Orchestration (Kubernetes)

5. Continuous Integration / Continuous Deployment (CI/CD)

- CI/CD Concepts
- Jenkins / GitHub Actions / GitLab CI / CircleCI
- Build Tools (Maven, Gradle, npm, etc.)
- Artifact Repositories (Nexus, Artifactory)

6. Cloud Providers

- AWS / Azure / GCP (choose one to start)
- Compute, Storage, Networking basics
- Cloud IAM & Security
- Serverless Concepts

7. Monitoring & Logging

- Monitoring Tools (Prometheus, Grafana, Datadog)
- Logging Tools (ELK Stack, Loki, Splunk)
- Alerting & Incident Response

8. Security (DevSecOps)

- Security Best Practices
- Secrets Management (Vault, AWS Secrets Manager)
- Vulnerability Scanning (Trivy, Clair)
- Compliance & Auditing

9. Networking & Service Mesh

- Service Discovery
- API Gateways (Kong, NGINX)
- Service Mesh (Istio, Linkerd)

10. Observability & SRE

- SLOs, SLIs, SLAs
- Distributed Tracing (Jaeger, Zipkin)
- Chaos Engineering

11. Soft Skills & Collaboration

- Communication & Collaboration
- Documentation
- Incident Management
- Project Management Basics

- Monorepo vs polyrepo tradeoffs at a high level
- Recovering disasters with `reflog`, `cherry-pick`, `git bisect`

Programming for Automation

- Pick one: Python 3.12+ with `uv` and `ruff`, or Go 1.23+
- Reading and writing JSON, YAML, TOML, HCL
- HTTP clients: `httpx` for Python, `net/http` for Go
- CLI building: `typer` or `click` for Python, `cobra` for Go
- Subprocess and pipeline patterns, capturing stderr properly
- Retries, timeouts, exponential backoff with jitter
- Structured logging, no `print` debugging in shipped code
- Writing a tiny library plus tests with `pytest` or `go test`

Containers and OCI Basics

- What a container actually is: namespaces, cgroups v2, overlay filesystems
- Docker Engine, Podman, and the OCI runtime spec
- Writing a `Dockerfile`: layers, cache, `.dockerignore`, multi stage builds
- Image hygiene: distroless, Chainguard images, Wolfi, slim variants
- Build tools: `docker buildx`, Buildah, BuildKit, Nixpacks
- Tagging, semver, immutable digests, `docker scout` or `trivy` scans
- Compose for local stacks, then graduate off it for prod
- Volumes, bind mounts, networks, healthchecks

> Skip: Docker Swarm and Vagrant. Use Kubernetes (kind, k3d, Minikube) for orchestration and Dev Containers or `devbox` for reproducible local environments because the ecosystem moved years ago.

CI/CD Foundations

- Pipeline mental model: source, build, test, scan, publish, deploy
- GitHub Actions: workflows, reusable workflows, matrix builds, OIDC to cloud
- Caching dependencies and Docker layers correctly
- Artifact publishing to OCI registries (GHCR, ECR, GAR, ACR)
- Branch protection, required status checks, environments with approvals
- Secrets handling: never in repos, OIDC over long lived keys
- Pipeline as code reviewed like application code

> Skip: classic Jenkins with Groovy pipelines. Use GitHub Actions, GitLab CI, or Argo Workflows because hosted runners plus OIDC eliminate most of the toil Jenkins solved.

Cloud Foundations (pick one provider deeply)

- IAM mental model: identities, principals, policies, least privilege
- Compute primitives: VMs, managed instance groups, spot/preemptible
- Object storage (S3, GCS, Azure Blob), lifecycle rules, versioning
- VPC, subnets, security groups, NAT gateways, private endpoints
- Managed databases overview: RDS, Cloud SQL, Azure SQL, Aurora
- DNS and edge: Route 53, Cloud DNS, Azure DNS, CloudFront, Cloud CDN
- Cost basics: tagging, budgets, alerts, savings plans vs reserved
- Cloud CLI plus SDK: `aws`, `gcloud`, `az`, paired with profiles and SSO

Web Servers, TLS, and the Edge

- Nginx vs Caddy vs Traefik: when each wins
- Reverse proxy patterns: TLS termination, gzip/brotli, rate limiting
- Let's Encrypt and ACME, wildcard certs, cert rotation
- Static site hosting on object storage plus CDN
- Web Application Firewall basics, OWASP Top 10 awareness
- HSTS, CORS, CSP headers, cookie security flags
- Health endpoints, readiness vs liveness signaling

Documentation and Communication

- README that someone can run in under five minutes
- Architecture diagrams with Mermaid, Excalidraw, or `d2`
- Runbooks: when paged, do this, in this order, with these commands
- Decision records (ADRs) under `docs/adr/`
- Incident write ups: timeline, impact, contributing factors, actions
- Asking good questions in async channels with full context

Project 1: **`homelab-toolbox`** Self hosted single node stack with Caddy, a dashboard, Uptime Kuma, and one app deployed via Docker Compose behind HTTPS. Must: TLS via Let's Encrypt, all configs in Git, daily restic backups to object storage, a one line `make up` to recreate.

Project 2: **`cli-deploy-bot`** A Python or Go CLI that takes a Git repo, builds a container, pushes to GHCR, and deploys to a remote VM over SSH using `docker compose`. Must: signed commits enforced, GitHub Actions pipeline, structured logs, retry with backoff on transient failures.

Done with Beginner when:

- [ ] Can debug a broken Linux service using `journalctl`, `ss`, and `strace` without copy pasting from a chatbot
- [ ] Can write a multi stage Dockerfile that produces a sub 50 MB image for a real app
- [ ] Can stand up a GitHub Actions pipeline that builds, tests, scans, and pushes a signed image on every PR
- [ ] Can explain what happens between `curl https://example.com` and a 200 response, layer by layer
- [ ] Can recover a botched Git history using `reflog` and `rebase -i` instead of recloning
- [ ] Can provision a VM, harden SSH, install one service, and front it with HTTPS without a tutorial open

---

Intermediate (~180 to 260 hrs)

Infrastructure as Code

- Terraform and OpenTofu: providers, state, backends, workspaces
- Module design: inputs, outputs, versioning, registry publishing
- Remote state with locking on S3/DynamoDB, GCS, or Terraform Cloud
- Drift detection, `plan` in CI on PRs, `apply` gated on merge
- Pulumi for teams that prefer real languages (TypeScript, Python, Go)
- Azure Bicep for Azure heavy shops, AWS CDK as an alternative
- Policy as code with OPA, Conftest, and `tfsec`/`checkov`
- Refactoring with `moved` blocks and import workflows

> Skip: Puppet and Chef for new projects. Use Ansible for last mile config and Terraform, OpenTofu, or Pulumi for resources because declarative cloud APIs replaced agent based config management.

Configuration Management and Image Building

- Ansible: inventories, roles, playbooks, `ansible-lint`, vaults
- Idempotency, handlers, tags, `check` mode in CI
- Packer or `image-builder` for golden VM images
- Dev Containers spec for reproducible developer environments
- Nix and `devbox` for hermetic toolchains
- Cloud init for first boot bootstrapping
- When to bake images vs configure at boot

Kubernetes Core

- Cluster anatomy: API server, etcd, scheduler, controller manager, kubelet
- Workload kinds: Deployment, StatefulSet, DaemonSet, Job, CronJob
- Networking: Services, Endpoints, Gateway API, Ingress, NetworkPolicy
- Storage: PV, PVC, StorageClass, CSI drivers, snapshot APIs
- Config: ConfigMaps, Secrets, projected volumes, downward API
- Pod design: probes, lifecycle hooks, init containers, sidecars
- Resource requests, limits, QoS classes, HPA, VPA, KEDA
- `kubectl` mastery: contexts, `kubens`, `k9s`, `stern`, `kubectl debug`
- Cluster bootstrap: kind, k3d, kubeadm, EKS, GKE, AKS basics

Helm, Kustomize, and Templating

- Helm 3 chart anatomy, values, dependencies, sub charts
- Writing your own chart with sensible defaults and a values schema
- Kustomize overlays, components, patches, generators
- When to combine Helm plus Kustomize (render then patch)
- Chart testing with `helm-unittest` and `chart-testing`
- OCI based chart distribution from any registry
- Pre install and post upgrade hooks, and why to avoid them

GitOps with Argo CD and Flux

- The GitOps mental model: cluster state mirrors Git, always
- Argo CD: Applications, ApplicationSets, Projects, sync waves
- Flux: Kustomizations, HelmReleases, image automation controllers
- App of apps vs ApplicationSet generators
- Multi environment promotion: dev to staging to prod via Git PRs
- Drift remediation, prune policies, self heal
- Secrets in GitOps: SOPS, Sealed Secrets, External Secrets Operator
- Progressive sync strategies and dependency ordering

CI/CD Pipelines at Scale

- Reusable workflows, composite actions, custom GitHub Actions runners
- Self hosted runners on Kubernetes with Actions Runner Controller
- GitLab CI parent/child pipelines, `needs:` DAGs
- Argo Workflows and Tekton for cluster native pipelines
- Dagger for portable pipelines in code
- Build caches that actually work: BuildKit, Bazel remote cache
- Hermetic builds, reproducible builds, provenance metadata
- Pipeline observability: timings, flakiness, MTTR for red builds

Observability Stack

- The three plus one: metrics, logs, traces, profiles
- OpenTelemetry: SDKs, Collectors, semantic conventions, OTLP
- Metrics with Prometheus, recording and alerting rules, PromQL
- Long term storage: Thanos, Mimir, Cortex, VictoriaMetrics
- Logs with Loki and LogQL, structured logging discipline
- Traces with Tempo or Jaeger, tail based sampling
- Continuous profiling with Pyroscope or Parca
- Dashboards in Grafana, dashboard as code with `grafonnet` or Terraform
- Alert routing with Alertmanager, on call with PagerDuty, Opsgenie, Grafana OnCall

> Skip: Nagios, Icinga, and Graphite for new builds. Use Prometheus plus Grafana plus OpenTelemetry plus Loki and Tempo because the LGTM stack is the de facto open standard in 2026.

Cloud Networking and Identity

- Hub and spoke vs mesh VPC topologies
- Private connectivity: PrivateLink, Private Service Connect, Private Endpoints
- Transit gateways, peering, Direct Connect, ExpressRoute, Interconnect
- Workload identity: IRSA, GKE Workload Identity, Azure Workload Identity
- OIDC federation from GitHub, GitLab, and Buildkite to clouds
- Service accounts vs human SSO via Entra ID, Okta, Google Workspace
- Zero trust ingress: Cloudflare Access, Tailscale, Teleport, Pomerium

Databases, Caches, and Stateful Ops

- Managed Postgres tuning: connections, `pgbouncer`, autovacuum
- Backup and restore drills, PITR, logical vs physical
- Migrations with `Atlas`, `Liquibase`, `Alembic`, or `golang-migrate`
- Redis and Valkey: data structures, persistence modes, eviction
- Running stateful workloads on Kubernetes with operators (CNPG, Zalando)
- Object storage as a database substrate (Iceberg, Delta, lakeFS)
- Read replicas, connection pooling, failover testing

Secrets and Identity Management

- HashiCorp Vault: KV v2, dynamic secrets, transit, PKI engines
- Cloud native: AWS Secrets Manager, GCP Secret Manager, Azure Key Vault
- External Secrets Operator for Kubernetes
- SOPS plus age or KMS for Git stored secrets
- Short lived credentials, no static cloud keys in CI ever
- Secret scanning in pipelines: `gitleaks`, `trufflehog`
- Rotation strategies and break glass procedures

Testing Infrastructure

- Terratest, `terraform test`, and `tofu test` for IaC
- `kubeconform`, `kube-linter`, `kubescape` for manifests
- Policy tests with Conftest and OPA bundles
- Smoke tests post deploy: synthetic checks, `k6`, `grpcurl`
- Chaos engineering basics with Litmus or Chaos Mesh
- Ephemeral preview environments per PR

Project 1: `gitops-platform-starter` A reusable repo that provisions an EKS, GKE, or AKS cluster via OpenTofu, bootstraps Argo CD, and deploys a sample app plus Prometheus, Grafana, Loki, and cert-manager. Must: OIDC from GitHub to cloud, External Secrets pulling from Vault or cloud KMS, signed images verified on admission, a one command tear down.

Project 2: `multi-env-promotion` A monorepo with `dev`, `staging`, `prod` overlays where merging to `main` triggers progressive rollout. Must: Helm chart per service, Argo CD ApplicationSet generating environments, PR preview namespaces auto created and destroyed, change failure rate tracked in Grafana.

Done with Intermediate when:

- [ ] Can write a Terraform or OpenTofu module others import without hand holding
- [ ] Can debug a misbehaving Pod from `kubectl describe` to root cause in under 15 minutes
- [ ] Can ship a service to production via a Git PR with zero `kubectl apply` from a laptop
- [ ] Can write a PromQL query that answers a real SLO question and turn it into an alert
- [ ] Can rotate a leaked secret across CI, cluster, and app in under an hour
- [ ] Can explain the exact path of a request from public DNS to a Pod, including identity hops

---

Advanced (~200 to 320 hrs)

Platform Engineering and Internal Developer Platforms

- IDP mental model: golden paths over guardrails over freeform
- Backstage: catalog, software templates, TechDocs, plugins
- Port and Cortex as managed IDP alternatives
- Crossplane and `kro` for control planes via Kubernetes APIs
- Service catalogs, scorecards, and production readiness reviews
- Paved roads: one approved stack with escape hatches
- Developer experience metrics: lead time, deploy frequency, change fail rate
- Self service infra via PRs against platform repos

Advanced Kubernetes

- Custom Resource Definitions, controllers, and the Operator pattern
- Writing operators with Kubebuilder or Operator SDK
- Admission control: ValidatingAdmissionPolicy, mutating webhooks
- Scheduling: taints, tolerations, topology spread, affinity, descheduler
- Pod Security Standards, seccomp, AppArmor, gVisor, Kata
- API priority and fairness, ETCD tuning, control plane sizing
- Cluster API for cluster lifecycle as code
- Karpenter and cluster autoscaler economics

Service Mesh and Modern Networking

- Sidecar mesh (Istio, Linkerd) vs sidecarless (Cilium Service Mesh, Istio Ambient)
- mTLS everywhere, SPIFFE and SPIRE for workload identity
- Traffic policy: retries, timeouts, circuit breaking, outlier detection
- Gateway API replacing Ingress: HTTPRoute, GRPCRoute, TLSRoute
- Multi cluster service discovery, east west gateways
- eBPF networking with Cilium, observability with Hubble
- When you do not need a mesh

DevSecOps and Software Supply Chain

- SLSA levels, in toto attestations, build provenance
- SBOM generation with Syft, vulnerability scanning with Grype, Trivy
- Image signing and verification with Sigstore (`cosign`, Rekor, Fulcio)
- Admission policy with Kyverno or Gatekeeper enforcing signed images
- Runtime security with Falco, Tetragon, and eBPF detections
- Container hardening: distroless, non root, read only root filesystem
- Secrets in CI: ephemeral OIDC, no static keys, masked logs
- DAST, SAST, dependency review, license scanning in pipelines
- Compliance frameworks: SOC 2, ISO 27001, FedRAMP, PCI DSS at a working level

Site Reliability Engineering

- SLI, SLO, SLA, error budgets, and budget burn policies
- Toil identification and elimination through automation
- Postmortems: blameless, action items with owners and dates
- Incident command, comms templates, severity ladders
- On call rotations, follow the sun, paging hygiene
- Capacity planning, headroom, predictable saturation
- Game days, chaos days, disaster recovery drills
- Reliability patterns: bulkheads, hedged requests, load shedding

Progressive Delivery and Release Engineering

- Argo Rollouts and Flagger for canary, blue/green, and analysis
- Feature flags with OpenFeature, Flagsmith, Unleash, LaunchDarkly
- Release trains vs continuous deployment for which teams
- Database migrations safely paired with rollouts (expand, contract)
- Shadow traffic and traffic mirroring for risky changes
- Rollback discipline: one click, tested, fast

Performance, Scale, and Cost

- Profiling under load with Pyroscope or Parca, flamegraphs
- Load testing with `k6`, `vegeta`, Gatling, Locust
- Autoscaling tuned to real signals (RPS, queue depth, latency, custom metrics)
- Right sizing with VPA recommendations and Karpenter consolidation
- Spot and preemptible workloads, interruption handling
- FinOps: showback, chargeback, unit economics per request
- Tools: OpenCost, Kubecost, CloudZero, Vantage, Cloudability
- Carbon awareness with `kepler` and region/grid mix scheduling

eBPF and Modern Linux

- What eBPF is and why it changed observability and security
- Cilium for networking, network policy, and service mesh
- Hubble for flow visibility
- Tetragon for runtime detections
- Pixie for instant Kubernetes observability
- Profiling system calls and TCP retransmits without restarts
- Writing a tiny eBPF program with `libbpf` or `bpftrace`

Multi Cluster, Multi Region, Hybrid, and Edge

- Federated identity, federated DNS, global load balancing
- Active/active vs active/passive tradeoffs, RPO/RTO targets
- Cluster API plus Crossplane for fleet management
- Argo CD ApplicationSets generating per cluster apps
- KubeEdge, K3s, and Talos at the edge
- Data residency, sovereignty, and cross region replication patterns
- Disaster recovery: backups (Velero), region failover drills

AI for DevOps and AIOps

- LLMs in pipelines: PR review, test generation, changelog drafting
- Coding agents in CI: GitHub Copilot agents, Cursor, Continue, Aider
- Anomaly detection on metrics and logs with managed AIOps
- LLM gateways and guardrails: model routing, rate limits, PII filters
- RAG over runbooks for incident copilots
- Vector stores on Kubernetes (Qdrant, Weaviate, Milvus, pgvector)
- GPU scheduling: device plugins, MIG, time slicing, DRA
- Cost and safety: token budgets, prompt injection defenses, eval gates

Building and Running a Platform Team

- Mission statement, customers, SLAs the platform itself commits to
- Roadmap shaped by user research, not vendor decks
- Versioning the platform, deprecations with timelines
- Office hours, embedded engineer rotations, paved road adoption metrics
- Internal docs that beat Stack Overflow for your stack
- Hiring ladder and rotation between SRE, platform, and product teams

Project 1: `paved-road-platform` A multi tenant platform on Kubernetes with Backstage as the front door, Crossplane provisioning cloud resources from a Git PR, golden path templates for a Python service and a Go service, Argo CD GitOps, OpenTelemetry baked in, Sigstore verified admission, and FinOps dashboards. Must: spin up a new service end to end in under 15 minutes from a template, automated SLO scaffolding per service, signed and attested images required by policy.

Project 2: `chaos-and-recovery-lab` A reference repo that runs continuous chaos against the `paved-road-platform` and proves recovery. Must: weekly chaos schedule with Chaos Mesh, automatic incident creation with timelines, blameless postmortem template generated from telemetry, restore-from-backup drill that completes under a documented RTO, public status page driven by real probes.

Done with Advanced when:

- [ ] Can design and defend a multi region Kubernetes platform with explicit RPO/RTO and failover drills
- [ ] Can extend Kubernetes with a CRD plus controller solving a real internal problem
- [ ] Can lead an incident as commander, run a blameless postmortem, and ship the follow ups
- [ ] Can produce, sign, and verify an SBOM attested build chain that passes a SLSA audit
- [ ] Can cut cloud spend by 20 percent or more without breaking SLOs, with receipts
- [ ] Can mentor a mid level engineer through their first production rollout with progressive delivery

---

Interview Prep

- Linux internals quickfire: page cache, OOM killer, cgroups v2, namespaces
- Networking deep dive: trace a TLS request, explain QUIC, debug intermittent 502s
- Kubernetes mechanics: how a Pod actually gets scheduled and started
- System design: design a CI/CD platform for 5,000 engineers
- System design: design a global multi tenant Kubernetes platform with tenancy isolation
- IaC scenario: design module structure, state layout, and PR workflow for a 100 service org
- Observability scenario: an SLO is burning, walk through diagnosis using metrics, logs, traces, profiles
- Security scenario: a leaked AWS key, what you do in the first 15 minutes
- Reliability scenario: a Postgres primary dies during peak traffic, write the runbook
- Cost scenario: explain how you would cut a 250k/month cloud bill by 30 percent
- DSA basics: graphs, heaps, hashing, big O for an SRE level bar
- Coding round: write a CLI that batches API calls with retries, backoff, and a rate limiter
- Behavioral: tell me about an incident you led, what you changed afterward
- Behavioral: a time you said no to a product team and how you kept the relationship

---

Specializations

Platform Engineering

- Backstage plugins, software templates, TechDocs at scale
- Crossplane Compositions, Configurations, and `kro`
- Internal API design for self service infra
- Golden paths, paved roads, escape hatches
- Service ownership models and on call boundaries
- DORA and SPACE metrics instrumentation
- Developer experience research and feedback loops
- Versioning, deprecation, and migration playbooks

Site Reliability Engineering

- SLO math, error budget policies, multi window multi burn alerts
- Capacity modeling and load testing as a habit
- Incident command and exec comms during sev1
- Postmortem facilitation and follow through
- Chaos engineering programs at organizational scale
- Toil budgets and yearly toil reduction targets
- Reliability for stateful systems: Postgres, Kafka, Redis
- Reliability culture: blameless, learning oriented, evidence based

Cloud and DevSecOps

- Threat modeling cloud workloads (STRIDE, attack trees)
- Identity perimeter design: SCIM, conditional access, just in time
- Network perimeter: zero trust, PrivateLink, service connect
- Supply chain hardening: SLSA, in toto, Sigstore, reproducible builds
- Runtime defense: Falco, Tetragon, eBPF detections, response automation
- Kubernetes hardening: PSS restricted, Kyverno baseline, NetworkPolicy default deny
- Compliance evidence automation for SOC 2 and ISO 27001
- Secrets lifecycle: issuance, rotation, revocation, audit

MLOps and AI Platform Engineering

- Model registry and dataset versioning with MLflow, DVC, lakeFS
- Training orchestration with Kubeflow, Ray, Metaflow, Flyte
- Serving with KServe, Triton, vLLM, TGI, Ollama for self host
- GPU scheduling on Kubernetes: device plugins, MIG, DRA, time slicing
- Vector DBs and RAG infrastructure (Qdrant, Weaviate, pgvector, Milvus)
- LLM gateways: Portkey, LiteLLM, Kong AI Gateway
- Eval pipelines, prompt registries, drift detection
- Cost and quota management for shared GPU pools

FinOps

- Tagging strategy and showback/chargeback that finance trusts
- Commitment based discounts: savings plans, CUDs, reserved instances
- Spot orchestration with Karpenter, interruption handling
- Right sizing pipelines fed by VPA recommendations
- Unit economics: cost per request, per tenant, per feature
- Anomaly detection on spend, automated guardrails
- Multi cloud cost normalization with OpenCost
- Carbon accounting alongside dollars

Observability Engineering

- OpenTelemetry collector pipelines and processors at scale
- High cardinality strategies, exemplar driven debugging
- Long term metrics with Mimir, Thanos, VictoriaMetrics
- Tail based sampling and trace based testing
- Continuous profiling with Pyroscope or Parca in production
- Dashboards as code, alerts as code, reviewed in PRs
- SLO platform: shared library, generators, burn rate policy
- Telemetry cost control, sampling, and retention tiers

Mental Models and Glossary

- **GitOps**: the desired state lives in Git, controllers reconcile reality to match; if it is not in Git, it does not exist.
- **Reconciliation loop**: observe, diff, act, repeat; the heart of Kubernetes and operators.
- **Cattle, not pets**: name and treat nodes and Pods as replaceable; if losing one ruins your day, you have a pet.
- **SLO before alert**: alert on user pain via burn rate, not on CPU; CPU is a clue, not a symptom.
- **Pull over push for secrets**: workloads fetch short lived creds via identity; long lived shared secrets are an outage waiting to happen.
- **Idempotency**: running the same change twice yields the same result; the foundation of safe automation.
- **Blast radius**: scope every change so the worst case stays small; permissions, regions, and rollouts all enforce this.
- **Toil**: manual, repetitive, automatable work with no enduring value; track it, then kill it.
- **Provenance**: a verifiable record of how an artifact was built; without it, supply chain claims are vibes.
- **Paved road**: the supported way to do a thing in your org, easier than the freeform alternative, with explicit escape hatches.

Resources

- Linux Foundation Certified SysAdmin and CKA curricula: use for structured Linux and Kubernetes operator fundamentals.
- Kubernetes official documentation and KEPs repo: use for canonical concepts and where the project is heading next.
- The Kubernetes Book by Nigel Poulton (2026 edition): use for a hands on tour that stays current.
- Site Reliability Engineering and The Site Reliability Workbook by Google: use for SLO discipline and incident culture.
- Seeking SRE edited by David Blank Edelman: use for diverse real world SRE perspectives beyond Google.
- Designing Data Intensive Applications by Martin Kleppmann: use for the data, distributed systems, and consistency foundation under any platform.
- Production Kubernetes by Josh Rosso et al.: use for opinionated patterns when running clusters for real teams.
- CNCF Landscape and TAG documents: use for picking projects and understanding maturity levels.
- roadmap.sh DevOps and Platform Engineering tracks: use for a checklist style ecosystem overview that complements this roadmap.
- HashiCorp Learn and OpenTofu docs: use for canonical Terraform and OpenTofu patterns.
- AWS, Google Cloud, and Microsoft Learn official paths for SA Pro, DevOps Engineer, and equivalent: use for cloud specific depth.
- Grafana Labs LGTM tutorials and OpenTelemetry docs: use for the modern observability stack end to end.
- Sigstore, SLSA, and CNCF TAG Security guides: use for software supply chain hardening that actually maps to audits.
- KubeCon and SREcon talks on YouTube (CNCF and USENIX channels): use for current war stories and design patterns from operators at scale.
- Learnk8s and KodeKloud labs: use for repetition heavy hands on practice when concepts are not sticking.
- DevOps Topologies (Skelton and Pais) and Team Topologies: use for org design that platform engineering depends on.
- The DORA State of DevOps reports: use for evidence based practices and metrics that map to outcomes.
- CNCF Slack, r/kubernetes, and the Platform Engineering community Slack: use for peer help and tracking what real teams are choosing in 2026.